A.D PATEL INSTITUTE OF TECHNOLOGY

DEPARTMENT OF INFORMATION TECHNOLOGY

MINI PROJECT REPORT ON INVENTORY AND STOCK MANAGEMENT SYSTEM

DATABASE MANAGEMENT SYSTEMS (DBMS)

Submitted By:

Ishita Dhariyani – 12502080601038

Liri Godhaniya – 12502080601033

Ditsa Barod - 12502080601027

Academic Year: [2026–2027]

Submission Date: 27-10-2026

CHAPTER 1: INTRODUCTION & SYSTEM REQUIREMENTS

1.1 Background

Inventory management is an important activity for businesses that deal with physical products. It involves maintaining records of available products, monitoring stock levels, managing suppliers, and recording stock received or issued. Efficient inventory management helps organizations avoid stock shortages, reduce unnecessary purchases, and maintain accurate records.

Traditional inventory management methods, such as handwritten registers and spreadsheets, can become difficult to maintain when the number of products and transactions increases. These methods may result in duplicate records, calculation errors, and difficulties in tracking changes in stock.

The Inventory and Stock Management System is designed to demonstrate how a relational database can organize inventory information systematically. The project uses MySQL to store product details, categories, supplier information, and stock transactions in related tables.

The database also includes views, stored procedures, and triggers to support reporting, stock validation, and automatic stock updates. A website prototype named StockPilot provides a user-friendly interface for inventory-related operations.

1.2 Scope

The scope of this project includes maintaining product records, organizing products by category, storing supplier information, recording stock-in and stock-out transactions, and identifying products whose stock levels fall below their reorder levels.

The database supports structured data storage and retrieval through SQL queries. Its views provide inventory reports, low-stock alerts, and transaction history. Stored procedures and triggers demonstrate how database operations can be organized and automated.

The StockPilot website prototype demonstrates the proposed user interface for inventory management. At present, the prototype uses browser Local Storage and is not connected to the MySQL database. Backend integration is planned as a further development step.

The project is intended primarily for academic demonstration and can serve as a foundation for a more complete inventory management application.

1.3 Hardware/Software Specifications

The following specifications describe the suggested requirements for developing and running the Inventory and Stock Management System.

Hardware Requirements

Component	Suggested Requirement
Processor	Intel Core i3 or equivalent
RAM	4 GB minimum; 8 GB recommended
Storage	2 GB or more of available space
Display	Standard monitor or laptop display
Input Devices	Keyboard and mouse

Software Requirements

Software	Purpose
Operating System	Windows or another compatible operating system
MySQL Server	Stores and manages the database
MySQL Workbench	Creates tables and executes SQL queries
Web Browser	Runs the StockPilot website prototype
HTML	Defines the structure of the website
CSS	Controls the appearance and layout
JavaScript	Implements client-side interactions
PHP	Planned backend technology for connecting the website to MySQL

Functional Requirements

1. The system should store product, category, and supplier details.
2. It should maintain current stock quantities.
3. It should record stock-in and stock-out transactions.
4. It should identify products below their reorder levels.
5. It should provide inventory and transaction reports.
6. It should maintain relationships between database tables.

Non-Functional Requirements

1. **Data integrity:** Invalid records should be restricted through appropriate constraints.
2. **Usability:** The website prototype should provide a clear and understandable interface.
3. **Reliability:** Database operations should maintain consistent inventory records.

4. **Maintainability:** Tables and SQL objects should be organized for easier modification.

These specifications are suggested development requirements and do not represent verified hardware measurements from a particular computer.

CHAPTER 2: DATABASE DESIGN

2.1 Requirements & Business Rules

The database design of the Inventory and Stock Management System is based on the need to maintain accurate and organized inventory information. The database separates product details, category information, supplier information, and stock transaction records into four related tables.

Functional Requirements

- Store and retrieve product information.
- Associate products with their respective categories.
- Maintain supplier contact information.
- Record stock entering and leaving the inventory.
- Identify products that need replenishment.
- Generate reports using SQL queries and database views.

Business Rules

1. Every product must have a unique product ID.
2. Each product must belong to a valid category.
3. A product may be associated with a supplier; the supplier reference is optional.
4. Each stock transaction must refer to an existing product.
5. Transaction quantities must be greater than zero.
6. Unit prices, current stock, and reorder levels must not be negative.
7. A stock-out transaction must not remove more units than are available.
8. A product is considered low-stock when its current stock is below its reorder level.

Database Integrity

Primary keys uniquely identify records, while foreign keys maintain relationships between tables. Unique constraints prevent duplicate category names. Check constraints help prevent invalid numeric values. Triggers are used to validate stock-out operations and update the product's stock after a transaction is recorded.

These rules help maintain consistency and reduce errors in inventory records.

2.2 ER Diagram

An Entity-Relationship (ER) diagram represents the entities in a database and the relationships between them. The Inventory and Stock Management System contains four main entities: Categories, Suppliers, Products, and Stock Transactions.

Entities and Relationships

- **Categories:** Stores category identification, category name, and description.
- **Suppliers:** Stores supplier identification and contact details.
- **Products:** Stores product details, price, current stock, and reorder level.
- **Stock Transactions:** Stores stock-in and stock-out activity for products.

Normalization is the process of organizing data into related tables to reduce redundancy and maintain data integrity.

First Normal Form (1NF): A table is in 1NF when each column contains atomic values and each record can be uniquely identified. In our project, product details and supplier details are stored in structured columns rather than combining multiple values in one field.

Second Normal Form (2NF): A table is in 2NF when it is in 1NF and every non-key attribute depends on the whole primary key. Each table in our design uses a single-column primary key, avoiding partial dependencies on composite keys.

Third Normal Form (3NF): A table is in 3NF when it is in 2NF and non-key attributes do not depend transitively on the primary key. Supplier details are stored separately in the suppliers table, while category details are stored in the categories table. The products table refers to these tables through foreign keys.

Conclusion: Separating categories, suppliers, products, and stock transactions helps reduce unnecessary duplication and improves consistency. The schema is designed to follow the principles of 3NF.

CHAPTER 3: IMPLEMENTATION & SQL SCRIPTS

Data Definition Language (DDL) commands define database structures such as databases and tables.

```
CREATE DATABASE inventory_management;

USE inventory_management;


CREATE TABLE categories (
    category_id INT PRIMARY KEY AUTO_INCREMENT,
```

```
    category_name VARCHAR(100) NOT NULL UNIQUE,
    description VARCHAR(255)
);

CREATE TABLE suppliers (
    supplier_id INT PRIMARY KEY AUTO_INCREMENT,
    supplier_name VARCHAR(100) NOT NULL,
    contact_person VARCHAR(100),
    phone VARCHAR(20),
    email VARCHAR(100),
    address VARCHAR(255)
);

CREATE TABLE products (
    product_id INT PRIMARY KEY AUTO_INCREMENT,
    product_name VARCHAR(100) NOT NULL,
    category_id INT NOT NULL,
    supplier_id INT NULL,
    unit_price DECIMAL(10,2) NOT NULL
        CHECK (unit_price >= 0),
    current_stock INT NOT NULL DEFAULT 0
        CHECK (current_stock >= 0),
    reorder_level INT NOT NULL DEFAULT 10
        CHECK (reorder_level >= 0),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (category_id)
        REFERENCES categories(category_id),
    FOREIGN KEY (supplier_id)
        REFERENCES suppliers(supplier_id)
```

```
);

CREATE TABLE stock_transactions (
   transaction_id INT PRIMARY KEY AUTO_INCREMENT,
   product_id INT NOT NULL,
   transaction_type ENUM('IN','OUT') NOT NULL,
   quantity INT NOT NULL CHECK (quantity > 0),
   transaction_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
   remarks VARCHAR(255),
   FOREIGN KEY (product_id)
      REFERENCES products(product_id)
);
```

These statements create the four tables and establish their relationships. Primary keys uniquely identify records, while foreign keys maintain referential integrity. Check constraints restrict invalid numerical values in MySQL versions that enforce them.

Data Manipulation Language (DML) commands are used to insert, retrieve, update, and delete records.

Inserting Sample Records

```
INSERT INTO categories (category_name, description)
VALUES
('Electronics', 'Electronic accessories'),
('Stationery', 'Office and writing supplies'),
('Furniture', 'Office furniture');

INSERT INTO suppliers (supplier_name, contact_person, phone)
VALUES
('ABC Supplies', 'Rahul Patel', '9876543210'),
('Office World', 'Amit Shah', '9876501234');
```

```
INSERT INTO products
(product_name, category_id, supplier_id,
 unit_price, current_stock, reorder_level)
VALUES
('Wireless Mouse', 1, 1, 500.00, 50, 10),
('Keyboard', 1, 1, 800.00, 30, 10),
('Notebook', 2, 2, 50.00, 15, 20),
('Office Chair', 3, 2, 2500.00, 10, 5),
('USB Cable', 1, 1, 150.00, 40, 10);
```

Retrieving Records

```
SELECT * FROM products;
```

This query displays all product records.

```
SELECT product_name, current_stock
FROM products
WHERE current_stock < reorder_level;
```

This query identifies products whose current stock is below the reorder level. In the sample data, Notebook is a low-stock product because it has 15 units against a reorder level of 20.

Updating a Product

```
UPDATE products
SET unit_price = 550.00
WHERE product_id = 1;
```

This query changes the price of the product with ID 1.

SQL joins combine related information stored in different tables.

INNER JOIN

```
SELECT p.product_name, c.category_name
FROM products p
INNER JOIN categories c
ON p.category_id = c.category_id;
```

This query displays each product with its corresponding category name.

LEFT JOIN

```
SELECT p.product_name, s.supplier_name
FROM products p
LEFT JOIN suppliers s
ON p.supplier_id = s.supplier_id;
```

A LEFT JOIN displays all products, including products that do not have a supplier assigned.

GROUP BY and Aggregate Functions

```
SELECT c.category_name,
    COUNT(p.product_id) AS total_products
FROM categories c
LEFT JOIN products p
ON c.category_id = p.category_id
GROUP BY c.category_id, c.category_name;
```

This query counts the products belonging to each category.

```
SELECT SUM(current_stock) AS total_units
FROM products;
```

The SUM function calculates the total number of units across all products.

Stock Transaction Query

```
SELECT p.product_name,
    t.transaction_type,
    t.quantity,
    t.transaction_date
FROM stock_transactions t
JOIN products p
ON t.product_id = p.product_id
ORDER BY t.transaction_date DESC;
```

This query displays stock transaction details along with product names, with the most recent transactions first.

These queries demonstrate how SQL can be used for reporting and inventory analysis.

A view is a virtual table created using an SQL query. It helps simplify frequently used queries and presents information in a convenient format.

1. Inventory Report

```
CREATE VIEW inventory_report AS
SELECT product_id, product_name,
    unit_price, current_stock, reorder_level
FROM products;
```

This view provides a summary of product prices, stock quantities, and reorder levels.

2. Low-Stock Alerts

```
CREATE VIEW low_stock_alerts AS
SELECT product_id, product_name,
    current_stock, reorder_level
FROM products
WHERE current_stock < reorder_level;
```

This view identifies products that need replenishment.

3. Transaction History

```
CREATE VIEW transaction_history AS
SELECT t.transaction_id, p.product_name,
    t.transaction_type, t.quantity,
    t.transaction_date, t.remarks
FROM stock_transactions t
JOIN products p
ON t.product_id = p.product_id;
```

This view combines transaction information with product names.

Using the Views

```
SELECT * FROM inventory_report;

SELECT * FROM low_stock_alerts;

SELECT * FROM transaction_history;
```

Advantages of views:

- Simplify complex queries.
- Improve consistency in reporting.
- Present selected information in an organized format.
- Help users retrieve frequently required information efficiently.

If these views already exist in the database, use CREATE OR REPLACE VIEW where supported instead of creating duplicate views.

Stored Procedures

A stored procedure is a set of SQL statements saved in the database and executed when called.

The add_stock() and remove_stock() procedures are intended to record stock-in and stock-out transactions.

The following is an illustrative implementation consistent with the trigger design described below.

```
DELIMITER //

CREATE PROCEDURE add_stock(
    IN p_product_id INT,
    IN p_quantity INT,
    IN p_remarks VARCHAR(255)
)
BEGIN
    IF p_quantity <= 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Quantity must be positive';
    END IF;

    INSERT INTO stock_transactions
        (product_id, transaction_type, quantity, remarks)
    VALUES
```

```
    (p_product_id, 'IN', p_quantity, p_remarks);
END//

CREATE PROCEDURE remove_stock(
  IN p_product_id INT,
  IN p_quantity INT,
  IN p_remarks VARCHAR(255)
)
BEGIN
  IF p_quantity <= 0 THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Quantity must be positive';
  END IF;

  INSERT INTO stock_transactions
    (product_id, transaction_type, quantity, remarks)
  VALUES
    (p_product_id, 'OUT', p_quantity, p_remarks);
END//

DELIMITER ;
```

Triggers

A trigger executes automatically when a specified database event occurs.

The BEFORE INSERT trigger checks whether sufficient stock is available for an outgoing transaction.

```
DELIMITER //

CREATE TRIGGER validate_stock_out
BEFORE INSERT ON stock_transactions
FOR EACH ROW
```

```
BEGIN
    DECLARE available_stock INT;

    IF NEW.transaction_type = 'OUT' THEN
        SELECT current_stock INTO available_stock
        FROM products
        WHERE product_id = NEW.product_id
        FOR UPDATE;

        IF available_stock IS NULL
          OR available_stock < NEW.quantity THEN
           SIGNAL SQLSTATE '45000'
           SET MESSAGE_TEXT = 'Insufficient stock';
        END IF;
    END IF;
END//

DELIMITER ;
```

The AFTER INSERT trigger updates the stock quantity once a transaction has been recorded.

```
DELIMITER //

CREATE TRIGGER update_stock_after_transaction
AFTER INSERT ON stock_transactions
FOR EACH ROW
BEGIN
    IF NEW.transaction_type = 'IN' THEN
        UPDATE products
        SET current_stock = current_stock + NEW.quantity
        WHERE product_id = NEW.product_id;
```

```
    ELSE
        UPDATE products
        SET current_stock = current_stock - NEW.quantity
        WHERE product_id = NEW.product_id;
    END IF;
END//

DELIMITER ;
```

The procedures insert transactions and allow the AFTER INSERT trigger to update the stock. They do not update stock independently, avoiding double updates.

Note: These are reference SQL examples. Verify them against the actual database before claiming successful execution.

CHAPTER 4: SCREEN LAYOUTS & TEST RESULTS

4.1 Interface/Form Screenshots

The StockPilot website prototype demonstrates an interface for inventory management.

1. **Dashboard:** Presents a summary of inventory information.
2. **Product Management:** Allows users to view, search, add, edit, and delete products in the prototype.
3. **Stock Movements:** Demonstrates stock-in and stock-out entry forms.
4. **Categories and Suppliers:** Organizes product categories and supplier details.
5. **Low-Stock Report:** Identifies products requiring replenishment.
6. **CSV Export:** Demonstrates exporting inventory information.

Insert actual screenshots in the following locations:

- [Insert StockPilot Dashboard Screenshot]
- [Insert Product Management Screenshot]
- [Insert MySQL Workbench Screenshot Showing Tables]

The website prototype currently uses browser Local Storage. Its connection to MySQL through a backend remains in progress.

4.2 Sample Output Reports/Query Results

The following table presents illustrative sample output, not verified live database results.

Product	Current Stock	Reorder Level	Status
Wireless Mouse	52	10	In Stock
Keyboard	30	10	In Stock
Notebook	15	20	Low Stock
Office Chair	10	5	In Stock
USB Cable	40	10	In Stock

Test Cases

Test	Expected Result	Status
Add valid stock	Stock increases	Not executed
Remove available stock	Stock decreases	Not executed
Remove excess stock	Transaction rejected	Not executed
Identify low stock	Notebook is listed	Expected
Insert invalid product reference	Foreign-key constraint rejects it	Not executed

These tests should be executed in MySQL Workbench before the report is submitted with actual test results.

CHAPTER 5: CONCLUSION & REFERENCES

5.1 Summary

The Inventory and Stock Management System demonstrates the application of relational database concepts to inventory management. The database is organized into four related tables: categories, suppliers, products, and stock_transactions. Primary keys, foreign keys, constraints, views, stored procedures, and triggers support data organization and stock management.

SQL queries enable product retrieval, category-wise analysis, transaction history reporting, and identification of low-stock products. The StockPilot prototype demonstrates how a website can provide an accessible interface for inventory operations.

The project provides practical experience in database design, SQL programming, relational relationships, and data integrity.

5.2 Limitations/Future Improvements

The current website prototype stores data in browser Local Storage and is not yet connected to MySQL. Therefore, website operations should not be treated as live database operations.

Future improvements include:

- Connecting the website to MySQL through a PHP backend.
- Adding user authentication and role-based access.
- Improving input validation and error handling.
- Implementing secure database access and regular backups.
- Testing stock operations and concurrent transactions.
- Deploying the system to a suitable server.